Question 1: Which of the following most accurately describes what happens to pooling layer parameters during backpropagation?

A: WRONG – Stride is a hyperparameter set before training not something learned from the data. Confusing hyperparameters with trainable parameters is common.
B: CORRECT – Pooling is simply a downsampling operation. There are no weights and biases and it's behaviour is totally controlled by the initial hyperparameters set before training.
C: WRONG – Convolutional layers have learnable weights (kernels). Since pooling layers are often placed after convolutional layers, this characterisation can be incorrectly assigned to pooling.

Question 2: During inference on a single image, which statistics does the BatchNorm layer use for normalisation?

A: WRONG – Since BatchNorm is applied per batch, it can be easily confused that the statistics are derived from the batch (even though the batch is one single image).
B: CORRECT – Moving mean and moving variance calculated during training are used solely for this purpose. In fact, these values are never used during training.
C: WRONG – Similarly to A, it can also be confused that the single image statistics are averaged with the moving mean and moving variance to obtain a new moving mean and moving variance.

Question 3: A deep learning network must classify whether each of five independent disease markers are present in a medical scan (any combination can be positive). Which output layer is most appropriate?

A: CORRECT – For multi-class classification where more than one class can be positive, the Sigmoid activation function is the function to use.
B: WRONG – Sounds practical but fundamentally doesn't work as Softmax. It can even be the case that no single class exceeds the 0.5 threshold as Softmax requires that all results are summed to 1.
C: WRONG – The statement that Softmax outputs 'a valid probability for each class' is technically true but misleading in multi-label contexts. The sum to 1 is what makes Softmax wrong here as we are treating them as dependent markers when in fact they should not be (also applies to B).

Question 4: A feature map of shape 7 × 7 × 512 is passed through a Global Average Pooling layer. What is the output shape?

A: WRONG – Channels remained unchanged in pooling operations but only spatial dimensions are reduced.
B: WRONG - 1D vectors of length 512 are common immediately after GAP once the trailing spatial 1 × 1 dimensions are squeezed away, which many frameworks do automatically. This is technically a reshaping of the correct answer and is the output shape that enters a classification head.
C: WRONG - Reducing everything to a single scalar would collapse all spatial and channel information, destroying the per-channel feature summaries that make GAP useful.
D: CORRECT - Global average pooling applies a filter of size nh × nw (here 7 × 7) independently to each channel, averaging all values in that channel's feature map down to a single scalar. The channel count nc = 512 is preserved. Therefore, the output is 1 × 1 × 512.

Question 5: Which property of GELU most clearly distinguishes it from both ReLU and Leaky ReLU?

A: CORRECT - ReLU and Leaky ReLU are both monotonic (never decreases as the input increases). On the other hand, GELU is non-monotonic because it dips slightly below zero after recovering.
B: WRONG – GELU can be negative and is not always non-negative.
C: WRONG – GELU is not monotonically increasing across all input but is non-monotonic.
D: WRONG – Technically the answer is True but it is not the property that distinguishes GELU.

Question 6: A deep network uses Sigmoid activations in its hidden layers and suffers from slow learning in early layers. Why would switching to Tahn be useful?

A: WRONG – Than is not computationally cheaper than Sigmoid and computes exponential functions.
B: CORRECT - A gradient signal four times larger at each layer compounds dramatically in deep networks, providing meaningfully stronger learning signal to earlier layers.
C: WRONG – Both Sigmoid and Tanh suffer from vanishing gradients but Tahn suffers from this a bit less. However, for large |x|, the Tanh gradient approaches zero.

Question 7: After training, which BatchNorm parameters are fixed and cannot adapt if the network is fine-tuned on a new dataset?

A: CORRECT – First initialised to 1 and 0 respectively, γ (scale) and β (offset) are learned parameters updated via backpropagation. During fine-tuning they continue to adapt just like any other weight. Similarly, the moving mean and variance also continue to be updated with new batches.
B: WRONG - γ (scale) and β (offset) are not fixed and be updated during fine-tuning.
C: WRONG – Both γ (scale) and β (offset) are learnable.
D: WRONG – Both γ (scale) and β (offset) are learnable.

Question 8: Which type of pooling operation was applied in this image?

A: WRONG – If average pooling (filter 2, stride 2) was applied, the result would be [2.75, 4.75, 1.75, 1.75].
B: CORRECT - Here we apply max pooling with size 2 and stride 2 as the filter size is a 2 x 2 filter and every time we are moving by 2 pixels.
C: WRONG – If global max pooling was applied, the result would be [8].
D: WRONG – If max pooling (filter 2, stride 1 was applied, the result would be [6, 6, 8, 6, 6, 8, 3, 2, 4].

Question 9: A memory-constrained model is trained with a batch size of 2. BatchNorm layers appears to hurt rather than help performance. What is the most likely explanation?

A: WRONG – Any batch size is compatible but smaller batch sizes are more prone to noise.
B: WRONG – Although the learning rate can be affecting the result, BatchNorm does not always improve performance regardless of batch size.
C: CORRECT – BatchNorm requires relatively large batch sizes to compute meaningful statistics used for inference. With small batch sizes, statistics computed are very noisy.
D: WRONG - γ and β should not be initialised differently to cater for the small batch size. The issue is simply the small batch size.

Question 10: A CNN for semantic segmentation (pixel-level labelling) uses aggressive max pooling (larger strides) after every convolutional block. What is the most well-grounded reason as to why this is not optimal?

A: CORRECT - Although aggressive application (large filter sizes and large slides), does sacrifice fine-grained spatial information for efficiency, the main reason is that max pooling is translation-invariant. This means that the local features within a window are summarised, making the network's output robust to small shifts or transformations of the input, but sacrificing pixel-level information.
B: WRONG - Average pooling can retain more overall information than max pooling because it considers all pixel values in a window rather than just the maximum. However, it is a matter of trade-off: average pooling smooths features and loses sharp details, while max pooling retains the strongest features.
C: WRONG – Large slides and filter sizes means information is summarised faster, leading to discarding of certain fine-grained features. This is not optimal but option A is more well-grounded in reason.